Programming for AI Lab – Assignment 1

Python practice programs for the **Lab of Programming for AI** course
BS Artificial Intelligence – Riphah International University, Sahiwal Campus

All solutions are written in a single Jupyter Notebook: `pg4ai_assignment_1.ipynb`

Contents

#	Program	Key concepts
1	Student Result System	lists, dictionaries, functions, loops, file handling
2	Sentence Analysis	strings, sets, sorting, f-strings, file handling
3	Shopping Bill	nested dictionaries, functions, conditions, file handling

#	Program	Key concepts
4	Calculator	functions, `while` loop, input validation
5	Restaurant Orders	dictionaries, tuples, lists, functions, `while` loop, file handling

---

Question 1 – Student Result System

- Takes input for **5 students**: name, roll number, and marks in Python, AI and Maths.
- Stores each student as a dictionary inside a list.
- A `result()` function calculates **total marks, percentage and grade**.
- Grading scale:

Percentage	Grade
80 and above	A
70 – 79	B
60 – 69	C
50 – 59	D

Percentage	Grade
Below 50	F

- Finds the student with the **highest percentage**.
- Saves all student records to `result.txt`.

Question 2 – Sentence Analysis

The `analyze(sentence)` function reports:

- Total words and total characters
- Longest and shortest word
- Total vowels
- Unique words
- Unique words in alphabetical order

The analysis is saved to `words-analysis.txt`.

Question 3 – Shopping Bill

- Takes **5 products** with name, price and quantity (stored in a nested dictionary).
- `calculate_Bill()` prints the total for each product and returns the complete bill.
- A **10% discount** is applied if the bill is greater than 10,000.
- Finds the **most expensive product** (by unit price).

- Saves the bill details to `Bill.txt`.

Question 4 – Calculator

- Separate functions for `add`, `subtract`, `multiply`, `divide` and `modulus`.
- Supported operations: `+`, `-`, `*`, `/`, `%`
- Handles **division by zero** safely.
- Runs in a loop until the user chooses not to calculate again.

Question 5 – Restaurant Orders

- The menu is stored in a dictionary; each item number maps to a `(name, price)` tuple:

No.	Item	Price (Rs.)
1	Burger	500
2	Pizza	800
3	Fries	250
4	Drink	150

- `show_menu()` prints the menu on every round.
- The user enters an **item number** and a **quantity**; entering `0` finishes the order.

- An invalid item number prints an error message and asks again.
- Each order is stored in a list as a dictionary (`item`, `qty`, `price`), where `price` is the line total (unit price × quantity).
- `calculate_total(order)` adds up all line totals to get the final bill.
- The bill (each item with quantity and price, then the **TOTAL**) is printed and **appended** to `orders.txt`, so earlier orders are kept.

Example bill:

```
Burger x 2 = Rs. 1000
Drink x 1 = Rs. 150
TOTAL: Rs. 1150
```

How to Run

1. Install Python 3 and Jupyter:

   ```
   pip install notebook
   ```

2. Clone this repository:

   ```
   git clone <your-repository-url>
   cd <repository-folder>
   ```

3. Start Jupyter and open the notebook:

```
jupyter notebook
pg4ai_assignment_1.ipynb
```

4. Run the cells in order, from top to bottom, and enter the inputs when prompted.

Output Files

Running the notebook generates these text files:

- `result.txt` – student records (Q1)
- `words-analysis.txt` – sentence analysis (Q2)
- `Bill.txt` – shopping bill summary (Q3)
- `orders.txt` – restaurant order bills (Q5)

Technologies Used

- Python 3
- Jupyter Notebook

Author

Name: Sahir Bano
Roll No: 0053
Section: FA 25 BS AI B